Benchmark des modèles locaux (Ollama, CPU) — étape 3

Piste : « 3 questions qui débloquent les briefs ». Mis à jour le 10/11/2026. Chiffres bruts : `benchmark/runs/<pc>.json` ; tableaux complets régénérés par `python benchmark/run_benchmark.py --rapport` → tableaux.md.

1. Ce qu'on mesure, et pourquoi

Tâche	Usage dans le produit	Cas de test	Mesures
Recherche : besoin du commercial en langage courant → bonne question de la banque (24 questions)	Le junior tape « je ne sais pas quand ils lancent » et retrouve la question qui marche	15 besoins, avec la rubrique et la ou les questions attendues	Bonne rubrique au rang 1, bonne question au **rang 1**, dans le **top 3** (les 3 questions montrées au junior), dans le top 5, MRR, temps par requête
Extraction (LLM) : note du rendez-vous précédent → rubriques du brief déjà connues, en JSON	Savoir quelles rubriques sont encore vides, pour choisir les 3 questions (entrée `rubriques_connues` de `POST /questions`)	15 notes, avec les rubriques attendues	JSON valide, précision, rappel, F1, notes avec rubrique inventée, notes avec valeur inventée, temps par note

Méthode :

- Les 15 cas (briefs_test.csv) ont été écrits **avant** le premier prompt et n'ont pas été modifiés ensuite.
- Les besoins sont volontairement formulés avec d'autres mots que les questions de la banque, pour tester la compréhension du sens.
- Réglages : Ollama sur `127.0.0.1`, `temperature 0`, graine 42, vecteurs de la banque en cache.
- Le TF-IDF sert de référence sans IA.

2. Contrainte matérielle : mon PC

	PC principal (le mien)
Processeur	AMD Ryzen 5 PRO 4650U (6 cœurs / 12 threads), carte graphique intégrée → **CPU uniquement**
RAM	15,3 Go au total, mais **0,6 à 1,8 Go libres** pendant les tests (navigateur, VS Code, API ouverts)
Disque C: (où Ollama range les modèles)	**0,3 Go libres sur 237 Go**
Disque D:	116 Go libres (pas encore utilisé par Ollama)
Système	Windows 11 Pro, Ollama 0.40.16, Python 3.11.9

Conséquences :

- **Disque :** `bge-m3` (~1,2 Go), `mxbai-embed-large` (~670 Mo), `qwen2.5:3b` (~1,9 Go) et `llama3.2:3b` (~2 Go) n'ont pas pu être téléchargés. Même `qwen2.5:0.5b` (~400 Mo) ne tient pas dans les 0,3 Go restants.
- **RAM :** avec moins de 2 Go libres, Windows déplace une partie de la mémoire sur le disque. Ça ralentit le LLM et rend les temps variables : entre deux relances identiques, le temps moyen par note est passé de 9,2 s à 11,4 s.
- **Pour aller plus loin :** les gros modèles sont testés par mes coéquipiers sur leur PC, avec le même script (colonnes « coéquipier » plus bas).

3. Résultats — recherche de la bonne question (15 cas)

Modèle	Taille	Bonne rubrique au rang 1	Bonne question au rang 1	Bonne question dans le top 3	Bonne question dans le top 5	MRR	Temps / requête	Indexation de la banque	Testé sur
TF-IDF (référence, sans IA)	0	40 %	33 % (5/15)	47 % (7/15)	53 %	0,44	**0,1 ms**	instantané	mon PC
nomic-embed-text	274 Mo	**53 %**	**47 % (7/15)**	**60 % (9/15)**	**73 %**	**0,56**	70–120 ms	2,4 s (une seule fois, puis cache)	mon PC
all-minilm	~46 Mo	*en attente*							mon PC, après libération du disque
bge-m3	~1,2 Go	*à remplir*							**PC d'un coéquipier**

Lecture : sur 15 besoins, nomic-embed-text met la bonne question dans les 3 proposées 9 fois, contre 7 pour le TF-IDF.

4. Résultats — extraction JSON des rubriques connues (15 notes)

« Garde-fou » : une règle simple, sans IA, ajoutée **après** le premier passage. Une rubrique ne compte que si sa valeur est non vide et si ses mots se retrouvent dans la note. Comme elle a été décidée en voyant ces résultats, elle **devra être confirmée sur les 30 cas de validation** de l'étape 4.

Modèle	Taille	JSON valide	Précision	Rappel	F1	F1 avec garde-fou	Notes avec rubrique inventée (brut → garde-fou)	Notes avec valeur inventée	Temps / note (moy. / max)	Chargement	Testé sur
qwen2.5:1.5b	986 Mo	**100 %**	0,44	0,92	0,59	**0,72**	100 % → 87 %	60 %	11,4 s / 17,9 s	11 s	mon PC
qwen2.5:0.5b	~400 Mo	*en attente*									mon PC, après libération du disque
qwen2.5:3b	~1,9 Go	*à remplir*									**PC d'un coéquipier**

5. Trois exemples concrets

Exemple 1 : les embeddings comprennent le sens, le TF-IDF compare les mots (cas T14).

- Besoin : « Quel public vise-t-on : les jeunes, les familles ou le haut de gamme ? »
- TF-IDF : il propose en n°1 Q02 (« création, rénovation ou extension de gamme ? »), une question hors sujet.
- nomic-embed-text : il propose en n°1 Q05 (« plutôt un public jeune, familial ou premium ? »), la bonne réponse. Le cas est réussi alors que « haut de gamme » et « premium » ne partagent aucun mot.

Exemple 2 : un piège de vocabulaire fait échouer les deux méthodes (cas T09).

- Besoin : « Quand est-ce que ça doit arriver dans les magasins ? »
- La bonne réponse est Q19 (« à quelle date le produit doit-il être en rayon ? »).
- nomic-embed-text : il propose Q17 (« à quel prix le produit fini sera-t-il vendu **en magasin** ? ») et ne place Q19 qu'au rang 15. Le mot « magasin » l'a attiré vers la mauvaise question.
- TF-IDF : il trouve la bonne rubrique, mais propose d'abord la question générique Q21 ; Q19 n'arrive qu'au rang 3.
- Leçon : il faudra tester une recherche **hybride** (TF-IDF + embeddings), et accepter que la recherche libre reste une aide, pas une vérité.

Exemple 3 : le petit LLM invente des rubriques (cas T03).

- Note : « Lancement prévu en septembre 2027 avec trois tours de validation. Ils n'ont pas encore d'idée précise sur le produit. » La seule rubrique attendue est `delai`.
- qwen2.5:1.5b coche les **7 rubriques**. Il invente par exemple `"cible": "femme"` et `"marche": "Europe"`, et prend le nom du client (« Ondevia ») pour une référence concurrente.
- Avec le garde-fou, il ne reste que `delai` et `references`. « Ondevia » est bien écrit dans la note, donc la règle ne peut pas détecter cette erreur de sens.

6. Recommandation pour la V2

1. **Recherche : nomic-embed-text, avec le TF-IDF en repli.** Il fait mieux sur toutes les mesures (+13 points au rang 1 et au top 3, +20 points au top 5) et reste rapide sur CPU (moins de 0,1 s par requête). Si Ollama ne répond pas, la V2 retombe automatiquement sur le TF-IDF. Une version hybride sera testée sur les 30 cas de validation.
2. **Extraction : pas d'extraction automatique avec qwen2.5:1.5b.** Même avec le garde-fou, il invente une rubrique dans 87 % des notes, et il faut plus de 11 s par note sur mon PC. Dans la V2, son résultat sert seulement de **pré-remplissage** : le junior confirme les rubriques connues en un clic, ce qui va dans le sens du produit (validation en un clic). Le garde-fou est appliqué systématiquement.
3. **Décision à prendre avec les résultats des coéquipiers :**
    - Si `qwen2.5:3b` atteint un F1 d'au moins 0,85 avec le garde-fou, en moins de 20 s par note, on le retient, sur une machine qui a la RAM nécessaire.
    - Si `bge-m3` dépasse nomic d'au moins 10 points sur le top 3, on le retient.
    - Sinon, on garde les modèles légers.

7. Limites (à dire au jury)

- **15 cas seulement** : un cas représente 6,7 points de pourcentage, donc les écarts de moins de 15 points ne sont pas significatifs.
- **Même auteur pour la banque et les cas de test** (données fictives) : risque de biais de formulation.
- **Le garde-fou a été conçu après avoir vu les résultats** → il faut le confirmer sur les 30 cas de validation, sans retoucher le prompt.
- **Temps mesurés sous forte contrainte de RAM**, sur un seul PC et un seul passage → ce sont des ordres de grandeur.
- **Couverture incomplète :** `all-minilm` et `qwen2.5:0.5b` ne sont pas encore testés (disque plein). `mxbai-embed-large` et `llama3.2:3b` sont abandonnés faute de place.
- Le TF-IDF et les embeddings ne voient que le texte des questions : la recherche ne tient pas encore compte du score de réussite des questions.

8. Reproduire

```
cd data-brique
ollama pull nomic-embed-text && ollama pull qwen2.5:1.5b      # + all-minilm, qwen2.5:0.5b si la place le permet
python benchmark/run_benchmark.py --pc pc_principal

# Sur le PC d'un coéquipier (étiquette sans nom de personne) :
ollama pull bge-m3 && ollama pull qwen2.5:3b
python benchmark/run_benchmark.py --pc pc2 --embed bge-m3 --llm qwen2.5:3b
# puis commiter benchmark/runs/pc2.json et régénérer les tableaux :
python benchmark/run_benchmark.py --rapport
```